Updated service

tasks.service.ts M

src > tasks.service.ts > TasksService > update

```ts
import { Injectable, NotFoundException } from '@nestjs/common';
import { InjectRepository } from '@nestjs/typeorm';
import { Repository } from 'typeorm';
import { Task } from './task.entity.js';

@Injectable()
export class TasksService {
  constructor(
    @InjectRepository(Task)
    private readonly tasksRepository: Repository<Task>,
  ) {}

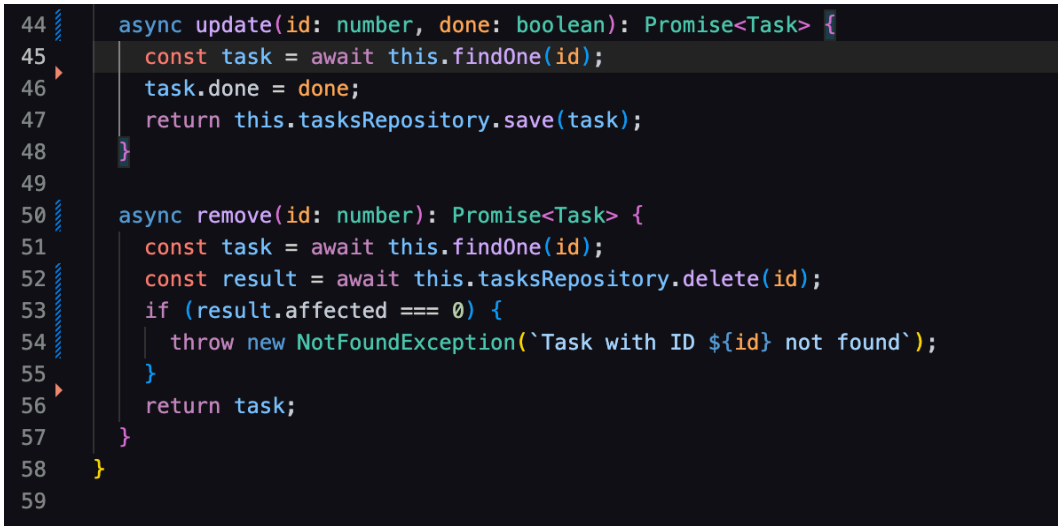
  async findAll(): Promise<Task[]> {
    return this.tasksRepository.find({ order: { id: 'ASC' } });
  }

  async findDone(): Promise<Task[]> {
    return this.tasksRepository.find({
      where: { done: true },
      order: { id: 'ASC' },
    });
  }

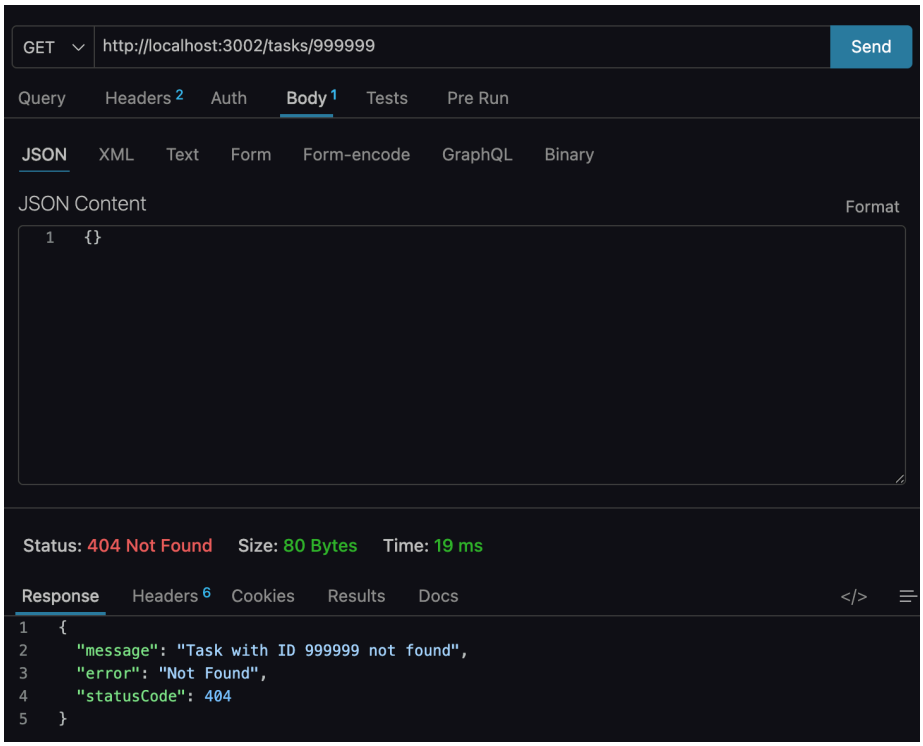
  async findTodo(): Promise<Task[]> {
    return this.tasksRepository.find({
      where: { done: false },
      order: { id: 'ASC' },
    });
  }

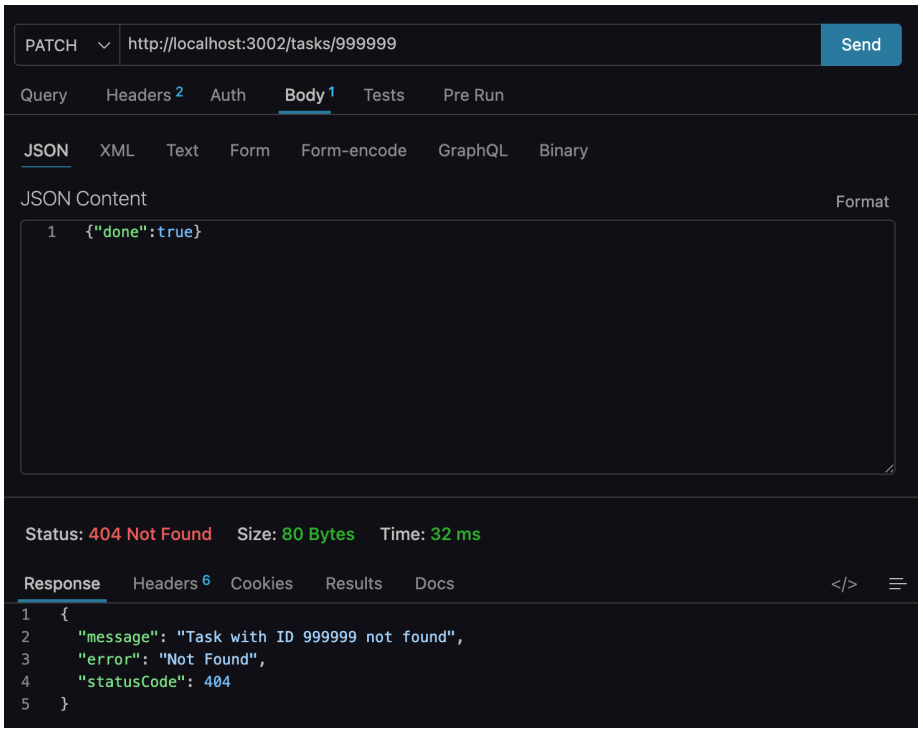
  async findOne(id: number): Promise<Task> {
    const task = await this.tasksRepository.findOneBy({ id });
    if (!task) {
      throw new NotFoundException(`Task with ID ${id} not found`);
    }
    return task;
  }

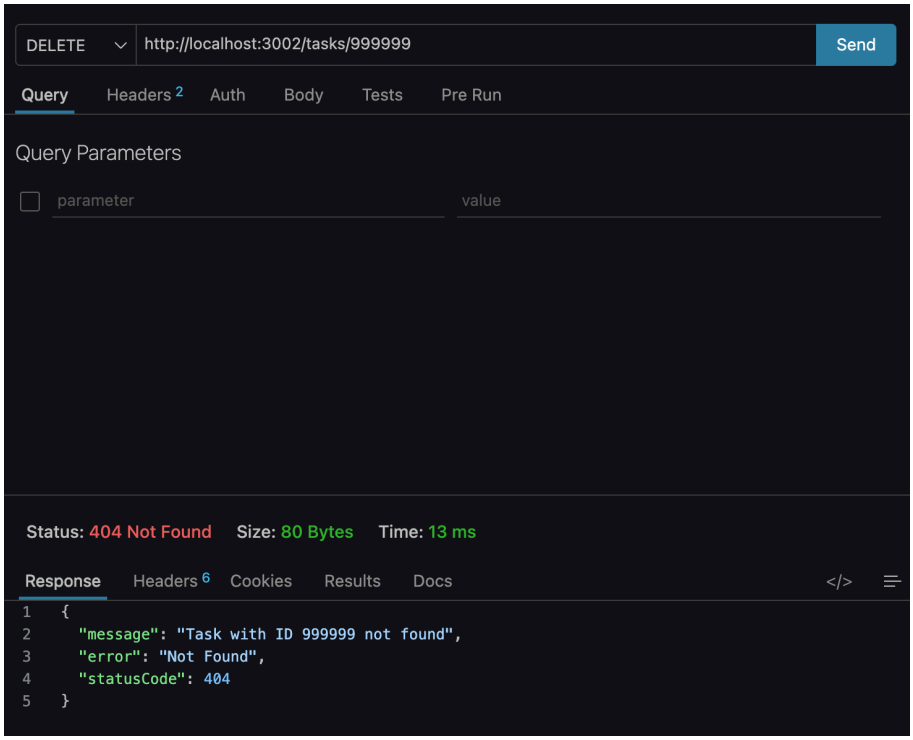
  async create(title: string): Promise<Task> {
    const task = this.tasksRepository.create({ title, done: false });
    return this.tasksRepository.save(task);
  }
```


async update(id: number, done: boolean): Promise<Task> {
const task = await this.findOne(id);
task.done = done;
return this.tasksRepository.save(task);
}
async remove(id: number): Promise<Task> {
const task = await this.findOne(id);
const result = await this.tasksRepository.delete(id);
if (result.affected === 0) {
throw new NotFoundException(`Task with ID ${id} not found`);
}
return task;
}
}

Screenshot the three 404 responses







Screenshot 400

